

TypeScript Problem-Solving Practice Task

Problem 1 — Gaming Age Rating Checker

Concepts: function parameter types, return types, string, conditional logic

Function name must be: `getAgeRatingAccess`

Scenario

A gaming platform, similar to Steam or the PlayStation Store, needs to determine which age-rating category of games a user is allowed to access, based on their age.

Task

Create a function named `getAgeRatingAccess`. The function should receive the user's age and return the highest age-rating category they can access.

Rating Rules

Age	Access Level
Below 7	"E (Everyone)"
7–12	"E10+ (Everyone 10+)"
13–16	"T (Teen)"
17 or above	"M (Mature)"

Function Requirement

- Accept the age as a **number**.
- Return the access level as a **string**.
- Correctly handle all age ranges.

Starter Code

```
function getAgeRatingAccess(age: number): string {  
  
    // write your code here  
  
}
```

Example

```
getAgeRatingAccess(5);  
// "E (Everyone)"  
  
getAgeRatingAccess(10);  
// "E10+ (Everyone 10+)"  
  
getAgeRatingAccess(15);  
// "T (Teen)"
```

```
getAgeRatingAccess(20);  
// "M (Mature)"
```

Problem 2 — Smart Home WiFi Signal Status

Concepts: function types, number, string, conditional logic, boundary conditions

Function name must be: `getSignalStatus`

Scenario

A smart home app shows the WiFi signal strength of each connected device as a percentage. Rather than showing a raw number, the app wants to display a friendly status label.

Task

Create a function named `getSignalStatus`. The function receives the signal strength (0–100) and returns a status string.

Status Rules

Strength (%)	Status
0–20	"Weak"
21–50	"Fair"
51–80	"Good"
81–100	"Excellent"

Additional Requirement

The function should return a `string`. Think carefully about boundary values such as:

Starter Code

```
function getSignalStatus(strength: number): string {  
    // write your code here  
}
```

Example

```
getSignalStatus(10);  
// "Weak"  
  
getSignalStatus(35);  
// "Fair"  
  
getSignalStatus(65);  
// "Good"  
  
getSignalStatus(95);  
// "Excellent"  
  
// boundary values to double-check:  
getSignalStatus(20);  
getSignalStatus(21);  
getSignalStatus(80);  
getSignalStatus(81);
```

Problem 3 — Esports Player Profile Card

Concepts: object types, type/interface, function parameter typing, return types, template literals

Function name must be: `formatPlayerCard`

Scenario

An esports tournament platform stores basic information about each registered player. The platform needs a function that turns that structured data into a readable profile card summary. A player contains `username`, `level`, and `region`.

Task

First define an appropriate TypeScript type or interface for the player. Then create a function named `formatPlayerCard` that receives a player object and returns a formatted sentence.

Requirements

- Accept a properly typed player object.
- Return a `string`.
- Use the values from the object rather than hard-coding the result.

Starter Code

```
// TODO: define a type or interface for the player  
  
function formatPlayerCard(player: Player): string {  
    // write your code here  
}
```

Example

```
formatPlayerCard({  
    username: "ShadowByte",  
    level: 42,  
    region: "Asia"  
});  
  
// Expected output:  
// "ShadowByte is a Level 42 player from Asia."
```

Problem 4 — Music App Listening Time Tracker

Concepts: object types, array types, `reduce()`, function parameter and return types

Function name must be: `calculateTotalListeningTime`

Scenario

A music streaming app wants to show users their yearly "Wrapped" summary — the total number of minutes they've listened to, based on an array of tracks they've played.

Track Structure

```
{
  title: string;
  minutes: number;
}
```

Task

Create a function named `calculateTotalListeningTime`. The function should receive an array of tracks and return the total minutes listened.

Requirements

- Properly type the track object.
- Properly type the array of tracks.
- Return the total as a `number`.
- An empty array should return `0`.

Starter Code

```
// TODO: define a type for a single track

function calculateTotalListeningTime(tracks: Track[]): number {

  // write your code here

}
```

Example

```
const tracks = [
  { title: "Blinding Lights", minutes: 3 },
  { title: "Levitating", minutes: 4 },
  { title: "Peaches", minutes: 3 }
];

calculateTotalListeningTime(tracks);
// 10

// another example:
const tracks2 = [
  { title: "Flowers", minutes: 3 },
  { title: "Anti-Hero", minutes: 4 }
];

calculateTotalListeningTime(tracks2);
// 7
```


Problem 5 — Esports Tournament Player Stats

Concepts: nested arrays, object types, `reduce()`, return object types, conditional logic

Function name must be: `getPlayerStats`

Scenario

A tournament platform stores each player's match scores in an array. Organizers want a quick summary with the player's name, average score, and rank tier. A player is ranked **MVP** if their average score is at least 80, otherwise **Rookie**.

Player Structure

```
{
  name: string;
  scores: number[];
}
```

Task

Create a function named `getPlayerStats`. The function should receive a player object, calculate the average of all scores, determine the rank, and return a new object containing `name`, `average`, and `rank`.

Edge Case

Think about what your function should do if the scores array is empty.

Starter Code

```
// TODO: define a type for a player

function getPlayerStats(player: Player): { name: string; average: number; rank: string } {
  {

    // write your code here

  }
}
```

Example

```
getPlayerStats({
  name: "Nova",
  scores: [90, 85, 95, 80]
});

// Expected output:
// { name: "Nova", average: 87.5, rank: "MVP" }

// another example:
getPlayerStats({
  name: "Zex",
  scores: [60, 55, 70, 50]
});

// Expected output:
// { name: "Zex", average: 58.75, rank: "Rookie" }
```

Problem 6 — Music App Feature Access

Concepts: union types, literal types, function parameter types, type safety

Function name must be: `canAccessOfflineMode`

Scenario

A music streaming app has three subscription tiers: `free`, `pro`, and `premium`. Only `pro` and `premium` subscribers are allowed to download songs for offline listening.

Task

First create a union type `Tier`, then create a function named `canAccessOfflineMode` that receives a valid `Tier` and returns whether that tier can use offline mode.

Rules

Tier	Offline Access?
free	false
pro	true
premium	true

TypeScript Requirement

This should produce a TypeScript error — `canAccessOfflineMode("student")` — since the purpose is to make TypeScript restrict the function to known tiers.

Starter Code

```
type Tier = "free" | "pro" | "premium";

function canAccessOfflineMode(tier: Tier): boolean {

    // write your code here

}
```

Example

```
canAccessOfflineMode("free");
// false

canAccessOfflineMode("pro");
// true

canAccessOfflineMode("premium");
// true
```

Problem 7 — Food Delivery Menu Filter

Concepts: typed arrays, object types, function parameters, filter(), return types

Function name must be: findMenuItemsByTag

Scenario

A food delivery app lets customers filter a restaurant's menu by a dietary tag, such as "vegan" or "spicy". Each menu item has a **name**, **price**, and a **tag**.

Task

Create a function named **findMenuItemsByTag**. The function should receive an array of menu items and a tag, and return all items that match that tag.

Requirement

If no item matches the tag, return an empty array.

Starter Code

```
// TODO: define a type for a menu item (including tag)

function findMenuItemsByTag(menu: MenuItem[], tag: string): MenuItem[] {

    // write your code here

}
```

Example

```
const menu = [
  { name: "Veggie Bowl", price: 350, tag: "vegan" },
  { name: "Tofu Wrap", price: 300, tag: "vegan" },
  { name: "Chicken Wings", price: 450, tag: "spicy" },
  { name: "Beef Burger", price: 500, tag: "spicy" }
];

findMenuItemsByTag(menu, "vegan");
// returns the Veggie Bowl and Tofu Wrap objects

findMenuItemsByTag(menu, "gluten-free");
// returns an empty array
```

Problem 8 — Notification Center Renderer

Concepts: union types, optional properties, type narrowing, discriminated unions, object types

Function name must be: `renderNotification`

Scenario

A social app's notification center receives two kinds of notifications: a **like** notification (someone liked your post) and a **system** notification (an announcement from the app). Each needs to be rendered as a different message.

Like Notification

```
{
  type: "like",
  fromUser: "Aisha"
}
```

System Notification

```
{
  type: "system",
  message: "Your subscription is expiring soon.",
  actionUrl: "/billing"
}
```

The `actionUrl` field is **optional** — a system notification may or may not include it.

Task

Create a function named `renderNotification`. The function should receive either a like notification or a system notification and return the rendered message string.

Rules

- A like notification renders as: `"<fromUser> liked your post."`
- A system notification renders as: `"System: <message>"`
- If a system notification has an `actionUrl`, append `" (Tap to view)"` to the end of the message.

TypeScript Requirement

Use a discriminated union (based on the `type` field) so TypeScript can correctly narrow between the two notification shapes.

Starter Code

```
// TODO: define types for LikeNotification and SystemNotification
// (remember: actionUrl on SystemNotification is optional)

function renderNotification(notification: LikeNotification | SystemNotification): string
{
    // write your code here
}
```

Example

```
renderNotification({ type: "like", fromUser: "Aisha" });
```

```
// "Aisha Liked your post."

renderNotification({ type: "system", message: "Maintenance complete." });
// "System: Maintenance complete."

renderNotification({ type: "system", message: "Your subscription is expiring soon.",
actionUrl: "/billing" });
// "System: Your subscription is expiring soon. (Tap to view)"
```

Problem 9 — Cloud Storage Usage Tracker

Concepts: union types, discriminated unions, type narrowing, object types, return types, conditional logic

Function name must be: `updateStorageUsage`

Scenario

A cloud storage app, similar to Google Drive, tracks how much storage a user has used. Every action a user takes is either an upload or a delete.

Upload Action

```
{
  type: "upload",
  sizeMB: 500
}
```

Delete Action

```
{
  type: "delete",
  sizeMB: 800
}
```

Task

Create a function named `updateStorageUsage`. It should receive the current storage used (in MB) and an action, and return the new storage usage.

Rules

- An **upload** increases the usage.
- A **delete** decreases the usage.
- Usage can **never go below 0** — if a delete would take it below 0, the result should just be `0`.

TypeScript Requirement

Represent the two possible action shapes using TypeScript's type system. The function should not accept arbitrary action types.

Starter Code

```
// TODO: define types for UploadAction and DeleteAction

function updateStorageUsage(currentUsageMB: number, action: UploadAction | DeleteAction):
number {

    // write your code here

}
```

Example

```
updateStorageUsage(2000, { type: "upload", sizeMB: 500 });
// 2500

updateStorageUsage(2000, { type: "delete", sizeMB: 800 });
// 1200

// usage floor at zero:
```

```
updateStorageUsage(300, { type: "delete", sizeMB: 1000 });  
// 0
```

Problem 10 — EV Scooter Rental Fee Calculator

Concepts: union types, function parameter types, return types, type narrowing, object-based configuration, conditional logic

Function name must be: `calculateRentalFee`

Scenario

A micromobility rental app offers three vehicle types: scooter, e-bike, and moped. The rental fee depends on the vehicle type and how many minutes it was rented for.

Pricing

Vehicle	Unlock Fee	Cost per Minute
scooter	10	2
ebike	15	3
moped	25	5

The final fee is calculated using: **Final Fee = Unlock Fee + (Minutes × Cost per Minute)**

Task

Create a function named `calculateRentalFee`. The function should receive a vehicle type and the rental duration in minutes, and return the calculated fee.

TypeScript Requirement

Create a union type so that only `"scooter" | "ebike" | "moped"` are accepted. This should be rejected by TypeScript: `calculateRentalFee("bike", 20)`.

Additional Challenge

Consider what should happen when `minutes` is `0`. Your function should still return a valid number based on your chosen interpretation of the pricing rules.

Starter Code

```
type VehicleType = "scooter" | "ebike" | "moped";

function calculateRentalFee(vehicle: VehicleType, minutes: number): number {
    // write your code here
}
```

Example

```
calculateRentalFee("scooter", 20);
// 50    (10 + 20 x 2)

calculateRentalFee("ebike", 20);
// 75    (15 + 20 x 3)

calculateRentalFee("moped", 20);
// 125   (25 + 20 x 5)
```

Recommended Solving Strategy

For every problem, follow this sequence:

Step 1 — Understand the Input

Ask: What exactly does my function receive?

```
function calculateTotalListeningTime(tracks: Track[]): number
```

Step 2 — Define the Data Types

If objects are involved, define their structure first.

```
type Track = {  
  title: string;  
  minutes: number;  
};
```

Step 3 — Define the Function

Clearly specify the parameter types and the return type.

```
function calculateTotalListeningTime(tracks: Track[]): number {  
  // solution  
}
```

Step 4 — Solve the JavaScript Logic

Don't overthink TypeScript. First determine the algorithm:

```
tracks  
  -> take each track's minutes  
  -> add them  
  -> return total
```

Step 5 — Use TypeScript to Make the Solution Safer

- Can this value have multiple types?
- Are there only a few valid values?
- Can a property be missing?
- What exactly should the function return?

Step 6 — Test Edge Cases

Don't only test the provided examples. For example:

```
calculateTotalListeningTime([]);  
  
findMenuItemsByTag(menu, "keto");  
  
getAgeRatingAccess(6);  
getAgeRatingAccess(7);  
getAgeRatingAccess(16);  
getAgeRatingAccess(17);
```

